

Rockchip RK3326 Linux SDK Quick Start

ID: RK-JC-YF-943

Release Version: V2.1.0

Release Date: 2023-12-20

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2023. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document describes the basic usage of the RK3326 Linux SDK, aiming to help developers master the RK3326 Linux SDK development package quickly.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Date	Version	Author	Revision History
2022-06-16	V1.0.0	WJL	Initial version.
2022-06-20	V1.8.0	WJL	Update version to V1.8.0
2022-09-20	V1.8.1	LinJianHua	Support Linux5.10 SDK
2022-11-20	V1.8.2	Caesar Wang	Update Linux Upgrade Instruction
2023-04-20	V1.9.1	WJL	Update the compilation instructions
2023-09-20	V2.0.0	WJL	Update instructions
2023-12-20	V2.1.0	WJL	Update instructions

Contents

Rockchip RK3326 Linux SDK Quick Start

1. SDK Precompiled Image
2. Set up an Development Environment
 - 2.1 Preparing the development environment
 - 2.2 Install libraries and toolsets
 - 2.2.1 Check and upgrade the `python` version of the host
 - 2.2.2 Check and Upgrade the `make` Version on the Host
 - 2.2.3 Check and Upgrade the `lz4` Version on the Host
 - 2.2.4 Check and Upgrade the `git` Version on the Host
3. Docker Environment Setup
4. Software Development Guide
 - 4.1 Development Guide
 - 4.2 Chip Information
 - 4.3 Buildroot Development Guide
 - 4.4 Third-Party OS Porting
 - 4.5 Software Update History
5. Hardware Development Guide
6. SDK Building Introduction
 - 6.1 SDK Project Directory Introduction
7. SDK Compilation Instructions
 - 7.1 One-Click Compilation
 - 7.2 SDK Compilation Command View
 - 7.3 SDK Board Level Configuration
 - 7.4 Compilation Commands
 - 7.5 Automated Compilation
 - 7.6 Individual Module Compilation
 - 7.6.1 U-boot Compilation
 - 7.6.2 Kernel Compilation
 - 7.6.3 Recovery Compilation
 - 7.6.4 Buildroot Compilation
 - 7.6.5 Cross-compilation
 - 7.6.5.1 Cross-compile toolchains inside SDK
 - 7.6.5.2 Built-in Cross Compilation in Buildroot
 - 7.6.6 Firmware Package
8. Upgrade Introdution
 - 8.1 Windows Upgrade Introduction
 - 8.2 Linux Upgrade Instruction
 - 8.3 System Partition Introduction

1. SDK Precompiled Image

By using the RK3326 Linux SDK precompiled image, developers can bypass the process of compiling the entire operating system from source code. Instead, they can directly flash the precompiled image into the RK3326 development board, enabling rapid start of development and related evaluations and comparisons. This can reduce development time and cost wastage caused by compilation issues.

The firmware can be downloaded from the public address [Click here for SDK firmware download](#).

Firmware path: General Linux SDK Firmware -> Linux5.10 -> RK3326

If you need to modify SDK code or for a quick start, please refer to the following chapters.

2. Set up an Development Environment

2.1 Preparing the development environment

It is recommended to use Ubuntu 22.04 for compilation. Other Linux versions may need to adjust the software package accordingly. In addition to the system requirements, there are other hardware and software requirements. Hardware requirements: 64-bit system, hard disk space should be greater than 40G. If you do multiple builds, you will need more hard drive space.

Software requirements: Ubuntu 22.04 or latest.

2.2 Install libraries and toolsets

When using the command line for device development, you can install the libraries and tools required for compiling the SDK by the following steps.

Use the following `apt-get` command to install the libraries and tools required for the following operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

Description:

The installation command is applicable to Ubuntu22.04. For other versions, please use the corresponding installation command according to the name of the installation package. If you encounter an error when compiling, you can install the corresponding software package according to the error message. in:

- Python 3.6 or later versions is required to be installed, and python 3.6 is used as an example here.
- make requires make 4.0 and above to be installed, take make 4.2 as an example here.
- lz4 1.7.3 or later versions is required to be installed.
- Git does not have the CVE-2022-39253 security detection patch.

2.2.1 Check and upgrade the `python` version of the host

The method of checking and upgrading the `python` version of the host is as follows:

- Check host `python` version

```
$ python3 --version
Python 3.10.6
```

If you do not meet the requirements of `python>=3.6` version, you can upgrade it in the following way:

- Upgrade `python 3.6.15` new version

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Check and Upgrade the `make` Version on the Host

The method to check and upgrade the `make` version on the host is as follows:

- Check the `make` version on the host

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrade to the new version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Check and Upgrade the `lz4` Version on the Host

The method to check and upgrade the `lz4` version on the host is as follows:

- Check the `lz4` version on the host

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
refusing to read from a console
```

- Upgrade to the new version of `lz4`

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

2.2.4 Check and Upgrade the `git` Version on the Host

- Check the `git` version on the host

```
$ /usr/bin/git -v
git version 2.38.0
```

- Upgrade to the new version of `git`

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

3. Docker Environment Setup

To help developers quickly complete the complex development environment preparation mentioned above, we also provide a cross-compiler Docker image. This enables customers to rapidly verify and then shorten the build time of the compilation environment.

Before using the Docker environment, you can refer to the following document for instructions:

`<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf`.

The following systems have been verified:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
ubuntu 22.04	24.0.5	pass	pass
ubuntu 21.10	20.10.12	pass	pass
ubuntu 21.04	20.10.7	pass	pass
ubuntu 18.04	20.10.7	pass	pass
fedora35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the website [Docker Image](#).

4. Software Development Guide

4.1 Development Guide

Aiming to help engineers get started with SDK development and debugging faster, *Rockchip_Developer_Guide_Linux_Software_EN.pdf* is released with the SDK, please refer to the documents under the project's `docs/en/RK3326` directory, which will be constantly improved and updated.

4.2 Chip Information

Aiming to help engineers get started with RK3326 SDK development and debugging faster, *Rockchip RK3326-S Datasheet V1.0-20220301.pdf* is released with the SDK, please refer to the documents under the project's `docs/en/RK3326/Datasheet` directory, which will be constantly improved and updated.

4.3 Buildroot Development Guide

To assist development engineers in quickly becoming familiar with the development and debugging of the Buildroot system, the *Rockchip_Developer_Guide_Buildroot_EN.pdf* development guide is released with the SDK. It can be found in the `docs/en/Linux/System` directory and will be continuously updated and improved.

4.4 Third-Party OS Porting

To help development engineers quickly become familiar with porting and adapting third-party OS, the *Rockchip_Developer_Guide_Third_Party_System_Adaptation_EN.pdf* development guide is released with the SDK. It can be found in the `docs/en/Linux/System` directory and will be continuously updated and improved.

4.5 Software Update History

- Software release version upgrade can be checked through project XML file by the following command:

```
# Linux4.4 SDK
.repo/manifests$ realpath rk3326_linux_release.xml
# e.g.:printf version v1.8.0, update time on 20220620
<SDK>/.repo/manifests/rk3326_linux_release_v1.8.0_20220620.xml

# Linux4.19 SDK
.repo/manifests$ realpath rk3326_linux4.19_release.xml
# e.g.:printf version v1.2.2, update time on 20221220
<SDK>/.repo/manifests/rk3326_linux4.19_release_v1.2.2_20221220.xml

# Linux5.10 SDK
.repo/manifests$ realpath rk3326_linux5.10_release.xml
# e.g.:printf version v1.4.0, update time on 20231220
<SDK>/.repo/manifests/rk3326_linux5.10_release_v1.4.0_20231220.xml
```

- The update content of software release version can be viewed through the engineering text, please refer to the engineering directory:

```
# Linux4.4 SDK
<SDK>/.repo/manifests/RK3326_Linux_SDK_Release_Note.md
or
<SDK>/docs/en/RK3326/RK3326_Linux_SDK_Release_Note.md

# Linux4.19 SDK
<SDK>/.repo/manifests/RK3326_Linux4.19_SDK_Release_Note.md
or
<SDK>/docs/en/RK3326/RK3326_Linux4.19_SDK_Release_Note.md

# Linux5.10 SDK
<SDK>/.repo/manifests/RK3326_Linux5.10_SDK_Release_Note.md
or
<SDK>/docs/en/RK3326/RK3326_Linux5.10_SDK_Release_Note.md
```

- The board version information can be obtained through the following methods:

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-5.10-gen-rkr7
ID=buildroot
VERSION_ID=2021.11
PRETTY_NAME="Buildroot 2021.11"
OS="buildroot"
BUILD_INFO="Tue Dec 26 16:39:24 CST 2023 - rockchip_rk3326_64"
KERNEL="5.10 - px30_linux_defconfig"
```

5. Hardware Development Guide

Please refer to user guides in the project directory for hardware development:

RK3326 hardware design guide:

```
<SDK>/docs/en/RK3326/Hardware/Rockchip_RK3326_Hardware_Design_Guide_V1.2_EN.pdf
```

RK3326 EVB hardware development guide:

```
<SDK>/docs/en/RK3326/Hardware/Rockchip_RK3326_User_Manual_EVB_V1.1_EN.pdf
```

6. SDK Building Introduction

6.1 SDK Project Directory Introduction

There are app, buildroot, device, docs, external, kernel, output, prebuilts, rkbin, tools, u-boot and other directories in the project directory. A manifest is used to manage repositories, and the repo tool is used to manage each directory or its subdirectories corresponding to git projects.

- app: Contains upper-level application APPs, mainly various application demos.
- buildroot: Root filesystem developed based on Buildroot (2021).
- device/rockchip: Contains chip board-level configurations as well as scripts and files for SDK compilation and firmware packaging.
- docs: Contains general development guidance documents, chip platform related documents, Linux system development related documents, and other reference documents.
- external: Contains third-party related repositories, including display, audio/video, camera, network, security, etc.
- kernel: Contains the code for Kernel development.
- output: Contains information about each generated firmware, compilation information, XML, host environment, etc.
- prebuilts: Contains the cross-compilation toolchain.
- rkbin: Contains Rockchip related binaries and tools.
- rockdev: Contains compiled output firmware, actually a symlink to output/firmware.
- tools: Contains commonly used tools under Linux and Windows operating systems.
- u-boot: Contains U-Boot code developed based on version v2017.09.

7. SDK Compilation Instructions

The SDK can be configured and compiled with `make` or `./build.sh` along with target parameters. For specific references, see the compilation instructions in `device/rockchip/common/README.md`.

To ensure SDK updates smoothly each time, it is recommended to clean previous compilation products before updating. This avoids potential compatibility issues or compilation errors, as old compilation products may not be suitable for the new version of the SDK. Run the following command to clean these compilation products:

```
./build.sh cleanall.
```

7.1 One-Click Compilation

Chip	Type	Reference Model	One-Click Compilation	rootfs Compilation	kernel Compilation	uboot Compilation
RK3326	Development Board	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_evb_lp3_v10_32bit_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v10-linux.img	./make.sh rk3326
RK3326	Development Board	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_evb_lp3_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v10-linux.img	./make.sh rk3326
RK3326	Development Board	RK3326 EVB LP3 V12	./build.sh lunch:rockchip_rk3326_evb_lp3_v12_32bit_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v12-linux.img	./make.sh rk3326
RK3326	Development Board	RK3326 EVB LP3 V12	./build.sh lunch:rockchip_rk3326_evb_lp3_v12_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 px30_linux_defconfig rk3326_linux.config make ARCH=arm64 rk3326-evb-lp3-v12-linux.img	./make.sh rk3326
RK3326	Development Board	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_robot64_defconfig&& ./build.sh	./build.sh rootfs	make ARCH=arm64 rk3326_linux_robot_defconfig make ARCH=arm64 rk3326-evb-lp3-v10-robot-linux.img	./make.sh rk3326
RK3326	Development Board	RK3326 EVB LP3 V10	./build.sh lunch:rockchip_rk3326_robot64_no_gpu_defconfig&& ./build.sh	./build.sh rootfs	make ARCH=arm64 rk3326_linux_robot_defconfig make ARCH=arm64 rk3326-evb-lp3-v10-robot-no-gpu-linux.img	./make.sh rk3326

Note:

- **Kernel, U-boot Compilation:** require specifying the toolchain.

For example, built-in 32-bit toolchain in the SDK:

```
export CROSS_COMPILE=./prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-
```

Built-in 64-bit toolchain in the SDK:

```
export CROSS_COMPILE=./prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
```

7.2 SDK Compilation Command View

`make help`, for example:

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
syncconfig          - Same as oldconfig, but quietly, additionally update
deps
olddefconfig        - Same as syncconfig but sets new symbols to their
default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

The actual operation of make is `./build.sh`

You can also run `./build.sh <target>` to compile the relevant functions, which can be done through

`./build.sh help` View the specific compilation commands.

```

$ ./build.sh -h

##### Rockchip Linux SDK #####

...

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]      choose chip
defconfig[:<config>]         choose defconfig
config                        modify SDK defconfig
...
updateimg                    build update image
otapackage                   build A/B OTA update image
all                           build all images
release                       release images and build info
save                          save images and build info
all-release                  build and release images
allsave                       build all images and save them
shell                         setup a shell for developing
cleanall                     cleanup
clean[:module[:module]]...   cleanup modules
...
post-rootfs <rootfs dir>      trigger post-rootfs hook scripts
help                          usage

Default option is 'all'.

```

7.3 SDK Board Level Configuration

Enter the project `<SDK>/device/rockchip/rk3326` directory:

Board level configuration	Note
rockchip_defconfig	Default
rockchip_rk3326_evb_lp3_v10_32bit_defconfig rockchip_rk3326_evb_lp3_v10_defconfig	Suitable for RK3326 EVB V10 development board with LP3/LP4
rockchip_rk3326_evb_lp3_v12_32bit_defconfig rockchip_rk3326_evb_lp3_v12_defconfig	Suitable for RK3326 EVB V12 development board with LP3/LP4
rockchip_rk3326_robot64_defconfig rockchip_rk3326_robot64_no_gpu_defconfig	Suitable for RK3326 Robot development board with tiny system

They can be configured through `make lunch` or `./build.sh lunch`:

```

$ ./build.sh lunch

##### Rockchip Linux SDK #####

...

Pick a defconfig:

```

```

1. rockchip_defconfig
2. rockchip_rk3326_evb_lp3_v10_32bit_defconfig
3. rockchip_rk3326_evb_lp3_v10_defconfig
4. rockchip_rk3326_evb_lp3_v12_32bit_defconfig
5. rockchip_rk3326_evb_lp3_v12_defconfig
6. rockchip_rk3326_robot64_defconfig
7. rockchip_rk3326_robot64_no_gpu_defconfig
Which would you like? [1]:
...

```

Other features can be configured through `make menuconfig`.

7.4 Compilation Commands

The SDK can be configured through `make menuconfig`, with the main components currently available for configuration being:

```

(rk3326) SoC
  Rootfs --->
  Loader (u-boot) --->
  Kernel --->
  Boot --->
  Recovery (based on buildroot) --->
  PCBA test (based on buildroot) --->
  Security --->
  Extra partitions --->
  Firmware --->
  Update (Rockchip update image) --->
  Others configurations --->

```

- Rootfs: This represents the "Root File System". Here, you can choose different root file systems like Buildroot, etc.
- Loader (u-boot): This is the boot loader configuration, usually u-boot, which initializes hardware and loads the main operating system.
- RTOS: Real-Time Operating System (RTOS) configuration options, suitable for applications requiring real-time performance.
- Kernel: Configure kernel options here, customizing a Linux kernel suitable for your hardware and application needs.
- Boot: Configure Boot partition support formats here.
- Recovery (based on buildroot): This is the configuration for the recovery environment based on buildroot, used for system recovery and upgrade.
- PCBA test (based on buildroot): This is a configuration for a PCBA (Printed Circuit Board Assembly) testing environment based on buildroot.
- Security: Indicates that certain security features will depend on the buildroot configuration chosen for Rootfs, mainly the Secureboot feature activation.
- Extra partitions: Used to configure additional partitions.
- Firmware: Configure firmware-related options here.
- Update (Rockchip update image): Used to configure options for Rockchip's complete firmware.
- Others configurations: Other additional configuration options.

The `make menuconfig` interface provides a text-based user interface to select and configure various options. Once configuration is complete, use the command `make savedefconfig` to save these settings, so that custom builds will be done based on these settings.

With the above configs, you can choose different Rootfs/Loader/Kernel configurations for various custom builds, allowing flexible selection and configuration of system components to meet specific needs.

7.5 Automated Compilation

To ensure smooth updates of the Software Development Kit (SDK) each time, it is recommended to clean previous compilation products before updating. This helps avoid potential compatibility issues or compilation errors, as old compilation products may not be suitable for the new version of the SDK. To clean these compilation products, run the command `./build.sh cleanall`.

Enter the root directory of the project and execute the following command to automatically complete all compilations:

```
./build.sh all # Compiles module code only (u-Boot, kernel, Rootfs, Recovery)
               # Then run `./build.sh ./mkfirmware.sh` to package the firmware

./build.sh     # Compiles module code (u-Boot, kernel, Rootfs, Recovery)
               # Packages into update.img complete upgrade package
               # Copies all compilation information and generates it in the out
directory
```

7.6 Individual Module Compilation

7.6.1 U-boot Compilation

```
./build.sh uboot
```

7.6.2 Kernel Compilation

- Method 1

```
./build.sh kernel
```

- Method 2

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-
x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 px30_linux_defconfig rk3326_linux.config
make ARCH=arm64 rk3326-evb-lp3-v10-linux.img -j
or
make ARCH=arm64 rk3326-evb-lp3-v12-linux.img -j
```

Note: `rk3326-evb-lp3-v10-linux` can be replaced with specific hardware dts

- Method 3

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 px30_linux_defconfig rk3326_linux.config
make ARCH=arm64 rk3326-evb-lp3-v10-linux.img -j
or
make ARCH=arm64 rk3326-evb-lp3-v12-linux.img -j
```

Note: `rk3326-evb-lp3-v10-linux` can be replaced with specific hardware dts

7.6.3 Recovery Compilation

Enter the root directory of the project and execute the following command to automatically complete the compilation and packaging of Recovery.

```
./build.sh recovery
```

After compilation, a recovery.img is generated in the Buildroot directory

`output/rockchip_rk3326_recovery/images`.

Note: recovery.img includes Kernel, so every time Kernel changes, Recovery needs to be repackaged. The method for repackaging Recovery is as follows:

```
source buildroot/envsetup.sh
cd buildroot
make recovery-reconfigure
cd -
./build.sh recovery
```

Note: Recovery is not a mandatory feature, some board configurations might not set it

7.6.4 Buildroot Compilation

Enter the root directory of the project and execute the following command to automatically complete the compilation and packaging of Rootfs:

```
./build.sh rootfs
```

After compilation, different formats of images are generated in the Buildroot directory

`output/rockchip_rk3326_64/images`, rootfs.ext4 format is used by default.

For more information, please refer to the Buildroot development document:

`<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Buildroot_EN.pdf`

7.6.5 Cross-compilation

7.6.5.1 Cross-compile toolchains inside SDK

Directory	Introduction
prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	gcc arm 10.3.1 64bit toolchain
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	gcc arm 10.3.1 32-bit toolchain

The toolchain can be downloaded from the following address:

[Click here](#)

7.6.5.2 Built-in Cross Compilation in Buildroot

Different chip and target feature configurations can be set via `source buildroot/envsetup.sh`

```
$ source buildroot/envsetup.sh
Top of tree: rk3326

Pick a board:

...
24. rockchip_rk3326_32
25. rockchip_rk3326_64
26. rockchip_rk3326_recovery
...

Which would you like? [1]:
```

Default choice is 25, `rockchip_rk3326_64`. Then, enter the RK3326 Buildroot directory to start compiling related modules.

`rockchip_rk3326_32` is for compiling a 32-bit Buildroot system, and `rockchip_rk3326_recovery` is used to compile the Recovery module.

For example, to compile the rockchip-test module, commonly used compilation commands are as follows:

- Enter the buildroot directory

```
SDK$ cd buildroot
```

- Delete and recompile rockchip-test

```
SDK$make rockchip-test-recreate
```

- Rebuild rockchip-test

```
SDK$make rockchip-test-rebuild
```

- delete rockchip-test

```
SDK$make rockchip-test-dirclean
or
SDK$rm -rf buildroot/output/rockchip_rk3326_64/build/rockchip-test-master
```

To compile an individual module or third-party application, a cross-compilation environment needs to be configured. For instance, for RK3326, its cross-compilation tools are located in `buildroot/output/rockchip_rk3326_64/host/usr`. You need to set the `bin/` directory of the tool and `aarch64-buildroot-linux-gnu/bin/` as environment variables and run the script for auto-configuring environment variables in the top-level directory:

```
source buildroot/envsetup.sh rockchip_rk3326_64
```

Enter the command to check,:

```
cd buildroot/output/rockchip_rk3326_64/host/usr/bin
./aarch64-linux-gcc --version
```

The following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot) 12.3.0
```

```
Save to the rootfs configuration file
buildroot$ make update-defconfig
```

7.6.6 Firmware Package

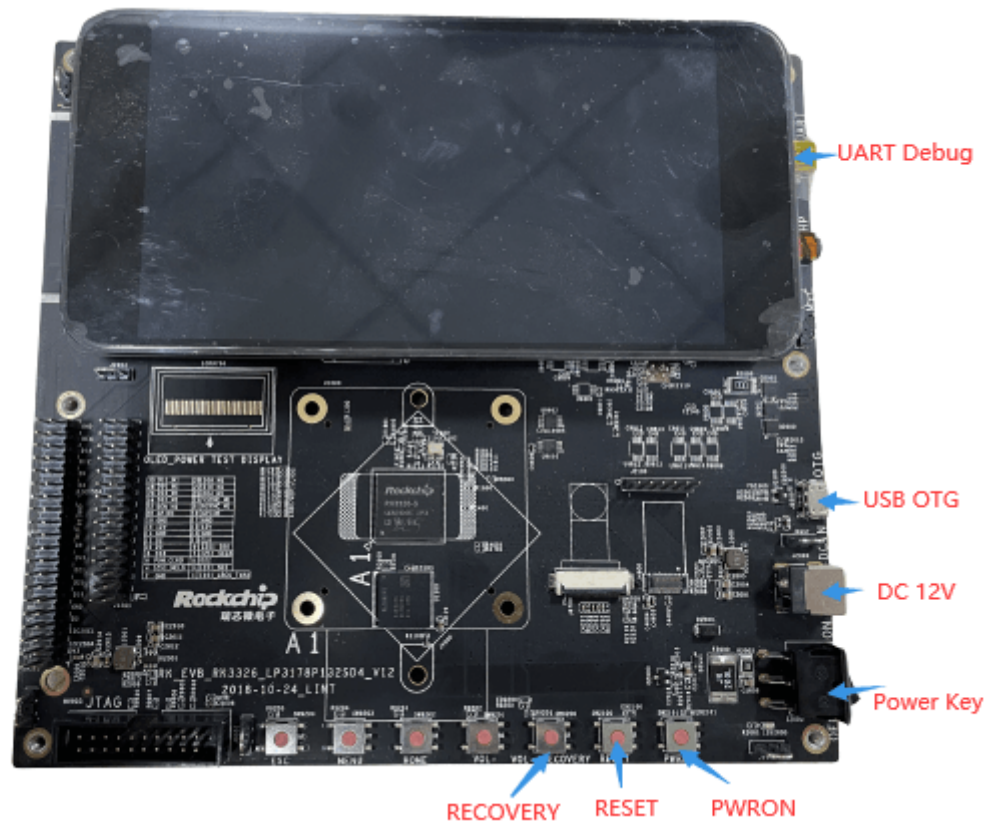
After compiling each part of Kernel, U-Boot, Recovery, Rootfs, enter the project root directory and execute the following command to automatically package all firmware to the `output/firmware` directory:

Firmware generation:

```
./build.sh firmware
```

8. Upgrade Introduction

Interfaces layout of RK3326 EVB board are showed as follows:

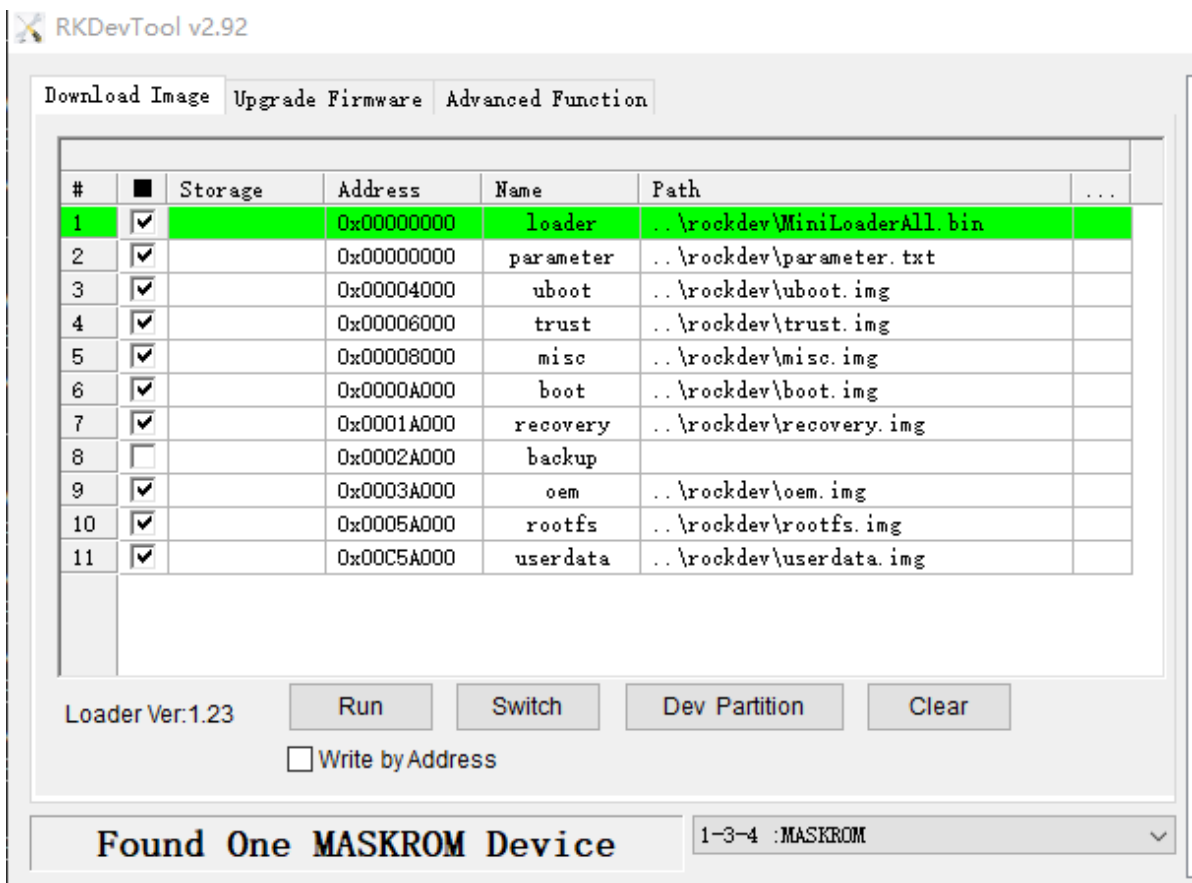


8.1 Windows Upgrade Introduction

SDK provides windows upgrade tool (this tool should be V2.92 or later version) which is located in project root directory:

```
tools/windows/RKDevTool
```

As shown below, after compiling the corresponding firmware, device should enter MASKROM or BootROM mode for update. After connecting USB cable, long press the button "MASKROM" and pressreset button "RST" at the same time and then release, device will enter MASKROM Mode. Then you should load the paths of the corresponding images and click "Run" to start upgrade. You canalso press the "recovery" button and press reset button "RST" then release to enter loader mode to upgrade. Partition offset and flashing files of MASKROM Mode are shown as follows (Note: Window PC needs to run the tool as an administrator):



Note: Before upgrade, please install the latest USB driver, which is in the below directory:

<SDK>/tools/windows/DriverAssitant_v5.12.zip

8.2 Linux Upgrade Instruction

The Linux upgrade tool (Linux_Upgrade_Tool should be v2.1 or later versions) is located in "tools/linux" directory. Please make sure your board is connected to MASKROM/loader rockusb, if the compiled firmware is in rockdev directory, upgrade commands are as below:

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

Or upgrade the whole update.img in the firmware

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or in root directory, run the following command on the machine to upgrade in MASKROM state:

```
./rkflash.sh
```

8.3 System Partition Introduction

Default partition introduction (below is RK3326 EVB reference partition):

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4096K	uboot
2	24576	32767	4096K	trust
3	32768	40959	4096K	misc
4	40960	106495	32M	boot
5	106496	303104	32M	recovery
6	172032	237567	32M	bakcup
7	237568	368639	64M	oem
8	368640	12951551	6144M	rootfs
9	12951552	30535646	8585M	userdata

- uboot partition: for uboot.img built from uboot.
- misc partition: for misc.img built from recovery.
- boot partition: for boot.img built from kernel.
- recovery partition: for recovery.img built from recovery.
- backup partition: reserved, temporarily useless. Will be used for backup of recovery as in Android in future.
- oem partition: used by manufactor to store their APP or data, mounted in /oem directory.
- rootfs partition: store rootfs.img built from buildroot.
- userdata partition: store files temporarily generated by APP or for users, mounted in /userdata directory.